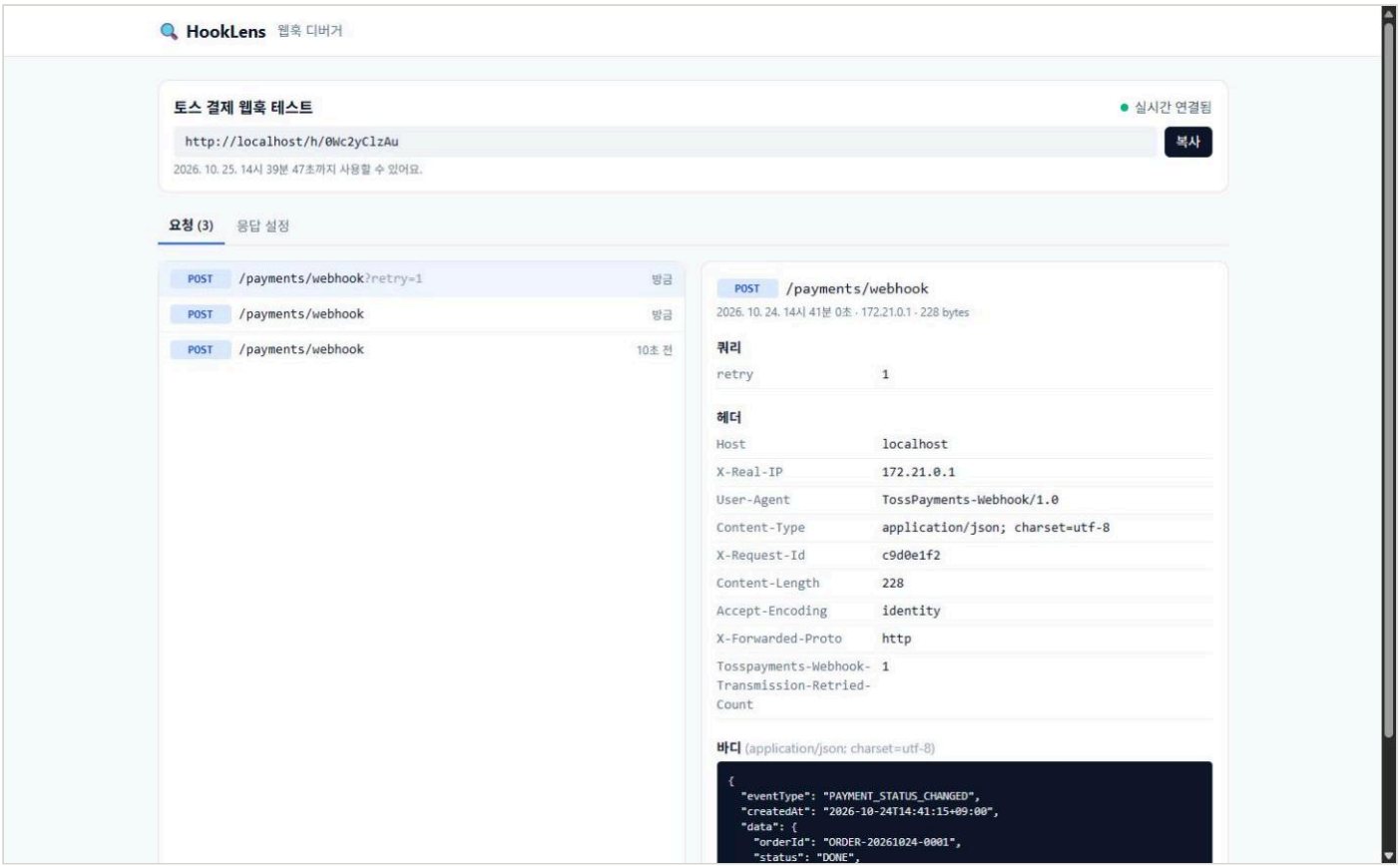
PERSONAL PROJECT · 2026.10

# HookLens

결제사·깃허브 웹훅을 받아서 실시간으로 보여주고,
정해 둔 응답을 돌려주는 웹훅 디버거

GITHUB github.com/songjeongseon1/hooklens



WHY

# 웹훅 연동을 만들 때 반복해서 막히는 세 가지

### 뭐가 오는지 모른다

결제사가 실제로 보내는 헤더와 바디를 문서만으로는 확신하기 어렵다.

**들어온 요청을 가공하지 않고** 메서드·경로·쿼리·헤더·바디·IP·시각까지 그대로 기록해서 보여준다.

### 내 PC로는 안 온다

결제사 서버는 인터넷에 있고, 개발 중인 localhost에는 웹훅을 보낼 수 없다.

버튼 한 번으로 **인터넷에서 접근 가능한 주소**를 발급한다. 가입 없이 24시간 동안 쓴다.

### 실패하면 어떻게 되나

우리 서버가 실패하거나 느릴 때 결제사가 재전송하는지 미리 볼 방법이 없다.

응답 코드·헤더·바디·**지연(0~5초)**을 정해 두면 다음 웹훅부터 그대로 응답한다.

---

**8 / 8**
MVP 요구사항 완료

**59**
테스트 (실제 MySQL 통합 테스트 포함)

**8**
기능별 PR · 커밋 26개

**10일**
설계부터 배포 구성까지

---

## 쓰는 흐름

**1. 발급**
이름만 넣고 버튼을 누르면 `/h/{slug}` 주소가 생긴다

→

**2. 등록**
결제사나 깃허브 설정의 웹훅 URL 칸에 붙여 넣는다

→

**3. 확인**
테스트 결제를 하면 요청이 화면에 바로 뜨고 헤더·바디를 본다

→

**4. 실패 재현**
응답을 503·3초로 바꿔 상대가 재전송하는지 본다

PLANNING

# 만들기 전에 범위부터 나눴다

### 요구사항 — 1차 MVP (P0)

| ID | 내용 | 상태 |
|---|---|---|
| EP-001 | 가입 없이 고유 주소 `/h/{slug}` 발급 | 완료 |
| EP-002 | 비회원 주소 24시간 후 만료, 이후 410 | 완료 |
| HOOK-001 | 모든 메서드·하위 경로 수신, 원본 그대로 저장 | 완료 |
| HOOK-002 | 설정한 응답 반환, 없으면 200 `{"ok":true}` | 완료 |
| LOG-001 | 새 요청 실시간 반영 (SSE) | 완료 |
| LOG-002 · 003 | 목록(최신순 커서) · 상세(쿼리·헤더·바디) | 완료 |
| RES-001 | 응답 설정 (상태 코드·헤더·바디·지연) | 완료 |

### 2차 (P1) · 이후 (P2)

| | | |
|---|---|---|
| P1 | 회원·JWT, 엔드포인트 관리, 분당 60건 제한(429), 바디 1MB 제한(413), 로그 보관 스케줄러, 리플레이·SSRF 차단 | 예정 |
| P2 | 조건부 응답, 로컬 서버로 자동 포워딩, cURL 내보내기, 검색 | 예정 |

### 나눈 기준

**"주소 만들기 → 받기 → 보기 → 응답 바꾸기"**가 끊김 없이 되는 것까지를 1차로 잡았다. 회원과 요청 제한은 계속 쓰려면 필요하지만, 없어도 핵심 흐름은 확인할 수 있어서 2차로 미뤘다.

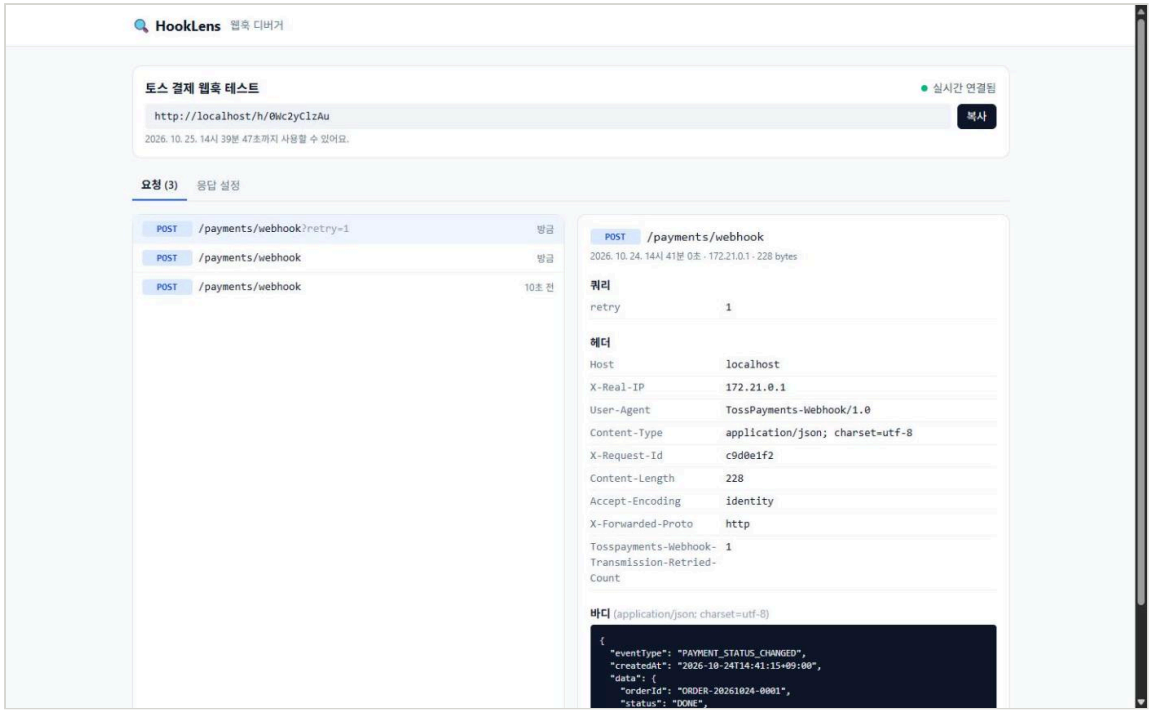
### 정책

| | |
|---|---|
| 관리 권한 | 발급 때 토큰을 한 번만 주고, 서버는 SHA-256 해시만 저장 |
| 만료 | 비회원 24시간 — 데이터가 무한히 쌓이지 않게 |
| 남의 요청 | 403이 아닌 404 — 존재 여부도 숨김 |
| 응답 지연 | 0~5초 — 길면 보낸 쪽이 타임아웃 |
| 응답 헤더 | 20개, 이름은 허용 문자만, 값에 줄바꿈 금지 (헤더 주입) |
| 시간 | 모든 계층 KST, API는 `+09:00` 포함 |

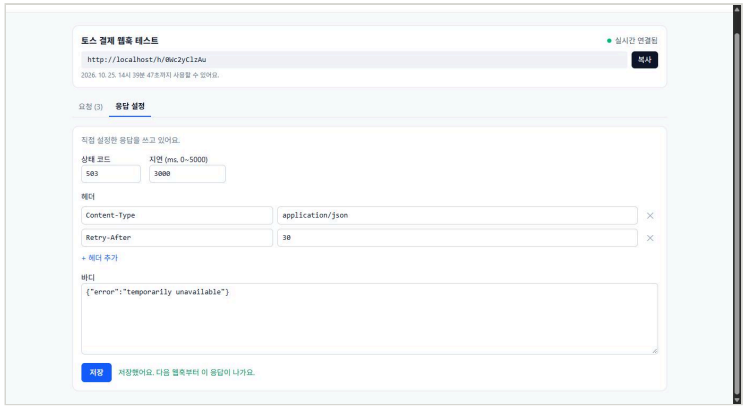
전체 기획서:
github.com/songjeongseon1/hooklens/blob/main/docs/planning.md

PRODUCT

# 새로고침 없이 들어오는 요청을 바로 본다




결제 웹훅 3건이 실시간으로 들어왔고, 재전송 요청의 헤더( `...Retried-Count: 1` )와 바디를 보고 있다.

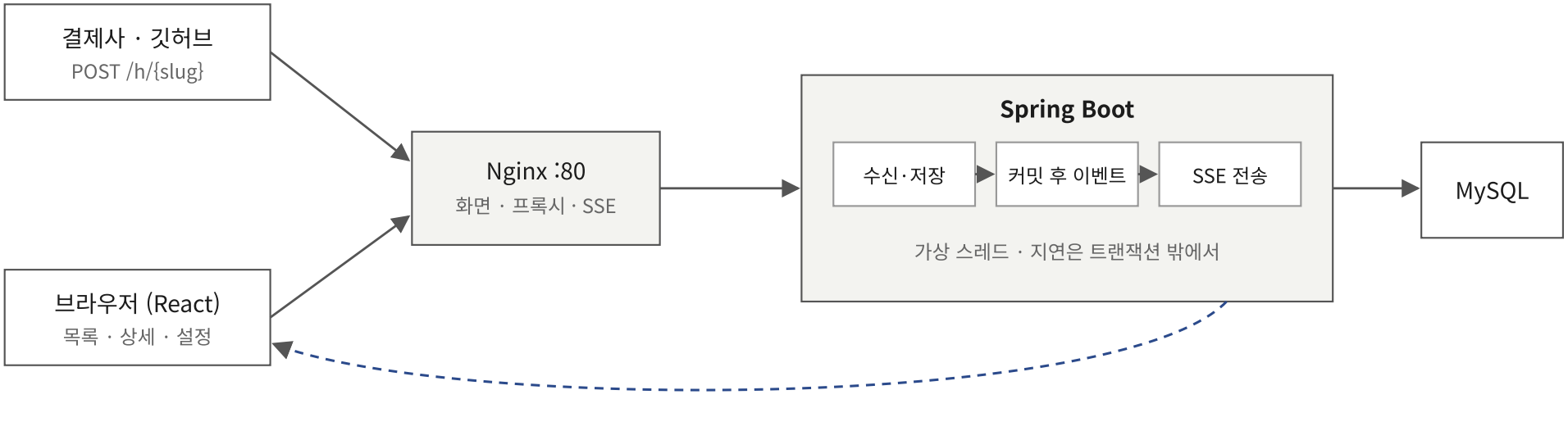



503 + 3초 지연 + `Retry-After` 로 바꾸면 다음 웹훅부터 이 응답이 나간다.

- 목록은 최신순 커서 페이지네이션
- 상세는 쿼리 표, 헤더 표, JSON 바디 정렬
- 로그인 대신 발급 때 한 번 받는 토큰으로 주인 확인

ARCHITECTURE

# 수신 · 저장 · 알림을 분리했다




### 수신은 알림을 모른다

저장 후 이벤트만 발행한다. 서버를 늘려 Redis Pub/Sub로 바꿔도 수신 코드는 그대로다.

### 밖으로는 80번만

Nginx 하나가 화면과 API를 같은 주소로 제공해 CORS 설정이 없다. 백엔드·DB 포트는 열지 않는다.

### 응답 형식 통일

관리 API는 `{success, data, error}`, 에러 코드는 enum 한 곳. 웹훅 수신만 사용자 설정대로 응답한다.

API

# API 7개, 응답 형식은 하나

| 메서드 | 경로 | 설명 | 인증 |
|---|---|---|---|
| POST | /api/endpoints | 엔드포인트 발급 | - |
| ANY | /h/{slug}/** | 웹훅 수신 | - |
| GET | /api/endpoints/{slug}/requests | 요청 목록 (커서) | 토큰 |
| GET | /api/endpoints/{slug}/requests/{id} | 요청 상세 | 토큰 |
| GET | /api/endpoints/{slug}/stream | 실시간 구독 (SSE) | 토큰 |
| GET | /api/endpoints/{slug}/response | 응답 설정 조회 | 토큰 |
| PUT | /api/endpoints/{slug}/response | 응답 설정 저장 | 토큰 |

### 인증 확인 순서 — 헤더 `X-Anonymous-Token`

**401** 토큰 없음 — DB 조회 전에 → **404** 없는 slug → **410** 만료됨 → **403** 토큰 불일치 — 시간 일정 비교

### 성공 · 실패 모두 같은 모양 (실제 응답)

```
{
  "success": true,
  "data": {
    "items": [{ "requestId": 6, "method": "POST",
                "path": "/payments/webhook", ... }],
    "nextCursor": null
  },
  "error": null
}
```

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "INVALID_INPUT",
    "message": "헤더 값에 줄바꿈을 넣을 수 없습니다: X-A"
  }
}
```

에러 코드는 enum 한 곳(`ErrorCode`)에서 관리한다. 웹훅 수신(`/h/**`)만 이 형식 대신 사용자가 정한 응답을 그대로 돌려준다.

요청·응답 필드와 에러 전체:
github.com/songjeongseon1/hooklens/blob/main/docs/api.md

DECISIONS · 1

# 실시간 알림을 안전하게 보내기

### SSE인데 EventSource를 쓰지 않았다

**문제** 관리 API는 헤더로 인증하는데 `EventSource`는 커스텀 헤더를 못 보낸다.

**비교** 쿼리 파라미터는 로그·기록에 토큰이 남고, 일회용 티켓은 저장소와 재발급이 필요하고, 쿠키는 인증이 둘로 갈린다.

**선택** `fetch`로 스트림을 직접 읽고 SSE를 파싱하는 훅. 끊기면 3초 뒤 재연결, 놓친 건 목록 API로 보충.

**결과** 인증 방식 하나, 토큰은 URL에 남지 않음.

### 알림은 커밋이 끝난 뒤에

**문제** 저장 중에 알리면 화면이 상세를 조회할 때 아직 커밋 전이라 404, 롤백되면 없는 요청을 알린다.

**선택** `@TransactionalEventListener(AFTER_COMMIT)`. 이벤트에는 엔티티 대신 DTO를 담아 트랜잭션 밖 LAZY 예외를 피함.

**결과** 실제 서버를 띄운 E2E 테스트로 확인. 리스너 전송을 지우면 테스트가 실패하는 것까지 확인했다.

DECISIONS · 2

# 느린 응답과 계속 늘어나는 목록

### 지연 응답이 커넥션을 잡지 않게

**문제** 최대 5초 지연을 트랜잭션 안에서 기다리면 커넥션 풀(10개)이 바닥나고, 톰캣 스레드도 묶인다.

**비교** 그냥 sleep / DeferredResult 비동기 / 가상 스레드

**선택** 로그를 먼저 커밋하고 컨트롤러에서 대기 + 가상 스레드. 코드는 sleep 그대로.

**결과** 지연 중에도 로그가 먼저 화면에 뜨는 것을 테스트로 고정. 스레드 이름 `tomcat-handler-N`으로 적용 확인.

### 보는 중에도 쌓이는 목록

**문제** 오프셋 페이징은 새 웹훅이 앞에 끼면 목록이 밀려 중복·누락이 생기고, 뒤로 갈수록 느리다.

**선택** `request_id < cursor` 커서 방식 + 같은 순서의 복합 인덱스. 다음 페이지는 COUNT 대신 `size + 1`개 조회로 판단.

**결과** 3건을 2건씩 나눠 보면 2 + 1, 중복 없음을 테스트로 확인.

DECISIONS · 3

# 로그인 없이 쓰는 서비스의 보안

### 남의 요청을 id로 훔쳐보기 (IDOR)

**문제** 주인 확인은 slug만 본다. 내 토큰 + 내 slug + **남의 요청 id**면 통과해 버린다.

**선택** `findByIdAndEndpointId`로 조건을 쿼리에 넣어 남의 것은 조회조차 안 되게. 403 대신 404로 존재 여부도 숨김.

**토큰** 32바이트 난수, DB엔 SHA-256 해시만. 비교는 `MessageDigest.isEqual`(타이밍 공격 방지).

### 사용자가 정한 응답 헤더 (헤더 주입)

**문제** 설정한 헤더가 응답에 그대로 나간다. 값에 `\r\n`이 있으면 `Set-Cookie` 같은 헤더를 끼워 넣을 수 있다.

**선택** 이름은 허용 문자 정규식, 값은 줄바꿈 금지. 어느 헤더가 왜 틀렸는지 400 메시지로. `Content-Length` 같은 전송 헤더는 무시.

**결과** 검사 호출을 빼면 테스트 2개가 실패하는 것을 확인.

QUALITY · DEPLOY

# 테스트는 실제 DB로, 배포는 같은 구성으로

## 테스트 59개

- H2 대신 Testcontainers로 **실제 MySQL**을 띄워 JSON 컬럼·인덱스·CASCADE까지 검증
- SSE는 실제 포트로 서버를 띄우고 `HttpClient`로 스트림에 붙어 이벤트 도착을 확인 (5초 제한)
- 시간은 `Clock` 빈에서만 가져와 만료 경계(1초 전·정각·1초 후)를 고정해 검증
- 보안 테스트는 검사 코드를 지웠을 때 실패하는지까지 확인

## Docker Compose 배포 구성

- Nginx + Spring Boot + MySQL, 밖으로는 80번만. 2단계 빌드로 실행 이미지 최소화(프론트 74MB)
- SSE 경로만 `proxy_buffering off`, SPA 새로고침은 `try_files`
- 운영은 `ddl-auto: validate` — 엔티티에서 만든 DDL을 MySQL 첫 실행 때 적용
- Nginx 뒤 실제 IP는 `X-Forwarded-For` + `forward-headers-strategy`
- 로컬에서 운영 구성 그대로 띄워 화면·SSE·IP·시간대 확인 (EC2 배포는 다음 단계)

RETROSPECTIVE

# 배운 것과 다음에 할 것

## 설정은 오류 없이 틀린다

`threads`를 `thread`로, `forward`를 `forword`로 쓴 적이 있다. Spring은 모르는 키를 조용히 무시해서 앱은 잘 뜨고 기능만 빠진다. 그 뒤로 설정을 바꾸면 스레드 이름, 저장된 IP처럼 **실제 동작을 눈으로 확인**한다.

## "왜"를 먼저 쓰고 코드를 썼다

기능마다 선택지를 비교하고 이유를 문서로 남긴 뒤 구현했다. 커서 페이징, AFTER_COMMIT, IDOR처럼 그냥 지나칠 뻔한 부분을 직접 고를 수 있었다.

## 아쉬운 점

범위를 잡을 때 프론트와 배포의 비중을 작게 봐서 EC2 배포까지는 마치지 못했다. 다음에는 배포 구성을 먼저 만들고 기능을 올리는 순서로 하려고 한다.

## 다음 단계

- EC2 배포, GitHub Actions 자동 배포
- 회원·JWT, 분당 요청 제한(Redis), 요청 리플레이와 SSRF 차단
- 다중 서버 SSE(Redis Pub/Sub), 스키마 이력 관리(Flyway)

코드 · 기술서 · 배포 절차 github.com/songjeongseon1/hooklens